1. Overview of Flutter and Dart

1.1 What is Flutter?

Flutter is an open-source UI software development framework developed by Google. It is used to develop cross-platform applications using a single codebase. Flutter supports Android, iOS, web, Windows, macOS, and Linux platforms.

1.2 Features of Flutter

1. ***Cross-platform development:*** Flutter allows developers to build applications for multiple platforms using a single codebase.
2. ***Hot Reload:*** It allows developers to see many code changes quickly without restarting the entire application.
3. ***Rich Widgets:*** Flutter provides various built-in widgets to create user interfaces.
4. ***Fast Development:*** It reduces development time by allowing code reuse.
5. ***Customizable UI:*** Developers can customize the appearance and behavior of UI components.
6. ***Open Source:*** Flutter is freely available and maintained by Google and its community.
7. ***Native Compilation:*** Flutter can compile applications into native machine code on supported platforms.

1.3 What is Dart? Dart is an object-oriented, class-based programming language developed by Google. It is the primary programming language used for Flutter application development. Dart is used to write application logic, define classes and functions, handle events, manage data, and create Flutter widgets.

Example:

```
void main() {
  String name = 'Nancy';
  int age = 23;

  print(name);
  print(age);
}
```

Output:

```
Nancy
23
```

Explanation:

- ***void:*** Specifies that a function does not return a value.
- ***main():*** The entry-point function of a Dart program.
- ***String:*** A data type used to store text.
- ***int:*** A data type used to store integer values.
- ***print():*** Displays output on the console.

1.4 Relationship Between Flutter and Dart: Flutter and Dart work together in application development. Dart is the programming language, while Flutter is the UI development framework. Dart is used to write the application code, and Flutter uses that code to build the user interface and application experience.

Overview of Flutter Architecture and How It Works

Introduction: **Flutter is an open-source UI development framework developed by Google. It is used to create cross-platform applications using a single codebase written in Dart. Flutter architecture consists of three main layers: The Framework Layer, the Engine Layer, and the Embedder Layer.**

1. Flutter Architecture

1. Framework Layer

- **Written in Dart.**
- **Provides widgets, layout, animations, gestures, and other UI components.**
- **Includes Material and Cupertino widgets.**
- **Helps developers build application interfaces.**

2. Engine Layer

- **Primarily written in C++.**
- **Handles graphics rendering, text layout, and low-level operations.**
- **Uses the Impeller rendering engine on supported platforms and configurations.**
- **Provides runtime support for Flutter applications.**

3. Embedder Layer

- **Connects the Flutter engine with the target operating system.**
- **Handles input events, application lifecycle, and rendering surfaces.**
- **Supports platforms such as Android, iOS, Windows, macOS, and Linux.**

2. How Flutter Works

1. **The developer writes application code using Dart.**
2. **Flutter widgets describe the user interface.**
3. **The framework converts widget descriptions into the required layout and rendering instructions.**
4. **The engine processes graphics and renders the interface.**
5. **The embedder connects the engine to the operating system.**
6. **The application displays the user interface and responds to user interactions.**

3. Important Features of Flutter Architecture

- **Widget-based: The user interface is built using reusable widgets.**
- **Cross-platform: A shared codebase can target multiple platforms.**
- **Hot reload: Many code changes can be viewed quickly during development.**
- **Fast rendering: Flutter uses its rendering engine to draw the interface.**
- **Customizable UI: Developers can create consistent and customized interfaces.**

Conclusion: **Flutter architecture combines the framework, engine, and embedder to develop and run applications across multiple platforms. This architecture provides flexibility, efficient development, and a consistent user interface.**

Setting Up the Flutter Development

Introduction **: Flutter is an open-source UI development framework developed by Google. Before developing Flutter applications, we need to install and configure the required software and tools.**

Steps to Set Up the Flutter Development Environment

1. Install Flutter SDK **Download the Flutter SDK from the official website: https://flutter.dev Extract the downloaded package to a suitable location on the computer.**

2. Configure Environment Variables **Add the Flutter SDK's bin directory to the system PATH environment variable. This allows Flutter commands to run from the terminal.**

3. Install Android Studio **Download and install Android Studio. It provides the Android SDK, emulator tools, and other utilities required for Android application development.**

4. Install Required SDK Components **Open Android Studio and install the required Android SDK platforms, SDK Build-Tools, and Android SDK Command-line Tools using the SDK Manager.**

5. Install an Editor **Use Android Studio or Visual Studio Code to write and manage Flutter applications. Install the Flutter and Dart plugins or extensions where applicable.**

6. Set Up an Emulator or Physical Device **Create an Android Virtual Device (AVD) using Android Studio's Device Manager, or connect a physical Android device with USB debugging enabled.**

7. Verify the Installation
Open a terminal or command prompt and run:

```
flutter doctor
```

This command checks the Flutter installation and identifies missing dependencies or configuration problems.

To create and run a sample application, use:

```
flutter create my_app
cd my_app
flutter run
```

Conclusion: **Setting up the Flutter development environment involves installing the Flutter SDK, configuring PATH, installing Android development tools, choosing an editor, and verifying the installation. Once these steps are completed, developers can create, test, and run Flutter applications.**

Dart Syntax and Data Types

Introduction: **Dart is an object-oriented, class-based programming language developed by Google. It is the primary programming language used for Flutter application development. Dart syntax defines the rules for writing programs, while data types specify the kind of values a variable can store.**

1. Dart Syntax

1. Main Function: The `main()` function is the entry point of a Dart program.

```
void main() {
  print("Hello Dart");
}
```

Output: Hello Dart

2. Variables: Variables are used to store data. Dart supports **`var`, `final`, `const`**, and explicitly declared types.

```
var name = "Nancy";
final age = 23;
const pi = 3.14;
```

3. Comments: Comments explain the code and are ignored during execution.

```
// Single-line comment
/*
Multi-line comment
*/
```

4. Operators: Dart supports arithmetic, relational, logical, and assignment operators.

```
int a = 10;
int b = 5;
print(a + b);
```

Output: 15

2. Data Types in Dart

1. int: Stores integer numbers.
```
int age = 23;
```
2. double: Stores floating-point numbers.
```
double price = 99.50;
```
3. String: Stores text.
```
String name = "Nancy";
```
4. bool: Stores Boolean values, either `true` or `false`.
```
bool isStudent = true;
```
5. List: Stores an ordered collection of values.
```
List<int> numbers = [10, 20, 30];
```
6. Set: Stores unique values without duplicates.
```
Set<int> numbers = {10, 20, 30};
```
7. Map: Stores key-value pairs.
```
Map<String, int> marks = {
        "Maths": 85,
        "Science": 90
    };
```
8. dynamic: Allows a variable to hold values of different types at runtime.
```
dynamic value = 10;
value = "Hello";
```

3. Type Inference

Dart can automatically determine a variable's type using the var keyword.

```
var city = "Rajkot";
```

Here, Dart infers city as a String.

Conclusion: Dart syntax provides rules for writing programs, and its data types allow developers to store and manipulate different kinds of data. These concepts are essential for developing Flutter applications efficiently.

Basic Flutter Widgets and Layout

Introduction

Flutter uses a widget-based architecture to build user interfaces. A widget is a basic building block that describes how a part of the user interface should appear. Widgets can represent text, buttons, images, rows, columns, and complete screens.

1. Basic Flutter Widgets

1. Text: Displays text on the screen.

```
Text('Hello Flutter')
```

2. Container: Provides a box that can have padding, margin, dimensions, decoration, and alignment.

```
Container(
  padding: EdgeInsets.all(10),
  color: Colors.blue,
  child: Text('Welcome'),      )
```

3. Row: Arranges widgets horizontally.

```
Row(

  children: [
    Text('Name'),
    Text('Nancy'),
  ],)
```

4. Column: Arranges widgets vertically.

```
Column(
  children: [
    Text('Flutter'),
    Text('Dart'),
  ],
)
```

5. Image: Displays an image from an asset or network.

```
Image.asset('assets/photo.png')
```

6. Icon: Displays a graphical symbol.

```
Icon(Icons.home)
```

7. ElevatedButton: Creates a raised button that responds to user interaction.

```
ElevatedButton(
  onPressed: () {},
  child: Text('Click Me'),
)
```

2. Flutter Layout

Flutter layout refers to the arrangement and positioning of widgets on the screen. Layouts are created by combining widgets in a widget tree.

Important layout widgets include:

1. Row: Arranges children horizontally.
2. Column: Arranges children vertically.
3. Container: Provides space, dimensions, alignment, and decoration.
4. Padding: Adds space around a child widget.
5. Center: Positions its child in the center of the available space.
6. Expanded: Makes a child of a Row, Column, or Flex fill the available space along the main axis.
7. Stack: Places widgets on top of one another.

3. Example of Flutter Layout

Start → Screen → Title → Body → Widgets

- `main()` = starting point
- `runApp()` = app launch
- `MaterialApp` = application wrapper
- `Scaffold` = screen structure
- `AppBar` = top bar
- `body` = main content
- `Column` = vertical arrangement
- `children` = multiple widgets

In this example, MaterialApp provides the application structure, Scaffold creates the screen layout, Center centers the content, and Column arranges the text and button vertically.

Conclusion

Flutter widgets are the fundamental building blocks of user interfaces. Layout widgets such as Row, Column, Container, Center, and Stack help developers arrange and position interface elements efficiently.

Basic Flutter Widget Properties and Methods

Flutter widgets have properties that define their appearance and behavior, and methods that perform specific operations.

1. Widget Properties

1. ***Text*: style changes text appearance, textAlign aligns text.**
2. ***Container*: width sets width, height sets height, padding adds inner space, margin adds outer space, color sets background color.**
3. ***Row and Column*: mainAxisAlignment aligns children along the main axis, crossAxisAlignment aligns them across the cross axis.**
4. ***ElevatedButton*: onPressed defines the action performed when clicked, child defines its content.**
5. ***Image*: width and height set image dimensions, fit controls how the image fits.**

2. Common Flutter Methods

- `setState()` **= updates state and requests a rebuild of a StatefulWidget.**
- `build()` **= describes the widget's user interface.**
- `initState()` **= initializes state when a State object is inserted into the widget tree.**
- `dispose()` **= releases resources when a State object is removed permanently.**
- `runApp()` **= starts the Flutter application by attaching the given widget to the screen.**

Common Methods

Properties = Appearance and behavior; Methods = Actions and operations.

- `Text` **= style and alignment**
- `Container` **= size and spacing**
- `Row/Column` **= alignment**
- `setState()` **= update**
- `build()` **= UI**
- `initState()` **= initialize**
- `dispose()` **= cleanup**
- `runApp( )` **= launch**

Conclusion: Widget properties control how widgets look and behave, while methods handle UI building, state updates, initialization, and cleanup.

Dart Libraries and Packages

Dart libraries and packages provide reusable code, functions, classes, and features that simplify application development.

1. Dart Libraries: **A library organizes related code and makes it reusable. The import keyword is used to access libraries.**

Example:

```
import 'dart:math';

void main() {
  print(sqrt(25));
}

Output: 5.0
```

2. Dart Packages: **A package is a collection of reusable code distributed for use in Dart and Flutter projects. Packages can provide additional functionality.**

Examples: http for network requests and provider for state management.

3. Types of Libraries

- **Core libraries = built-in features, e.g. dart:math**
- **Flutter libraries = UI development, e.g. package:flutter/material.dart**
- **External libraries = additional functionality from packages**

4. Managing Packages: Dependencies are declared in the pubspec.yaml file.

Example:

```
dependencies:
http: ^1.2.0
```

Run the following command to install dependencies:

```
flutter pub get
```

5. Advantages

- **Code reusability: Existing code can be reused in multiple applications.**
- **Reduced development time: Ready-made libraries reduce coding effort.**
- **Easier maintenance: Centralized code makes updates easier.**
- **Additional features: Packages provide functionality without building everything from scratch.**

Conclusion: Dart libraries and packages help developers organize code, reuse existing functionality, and develop Flutter applications efficiently.

Control Flow and Loops in Dart

Control flow statements determine the order in which program statements execute. Loops are used to execute a block of code repeatedly.

1. if Statement: Executes code when a condition is true.

```
if (age >= 18) {
  print("Adult");
}
```

2. if-else Statement: Executes one block if the condition is true and another if it is false.

```
if (age >= 18) {
  print("Adult");
} else {
  print("Minor");
}
```

3. switch Statement: Selects a code block based on a matching value.

```
switch (day) {
  case 1:
    print("Monday");
    break;
  default:
    print("Invalid day");
}
```

4. for Loop: Repeats code a specified number of times.

```
for (int i = 1; i <= 5; i++) {
  print(i);
}
```

5. while Loop: Repeats code while a condition is true.

```
int i = 1;
while (i <= 5) {
  print(i);
  i++;
}
```

6. do-while Loop: Executes code at least once before checking the condition.

```
int i = 1;
do {
  print(i);
  i++;
} while (i <= 5);
```

7. break and continue

- `break` = terminates the loop.
- `continue` = skips the current iteration and proceeds to the next.

Conclusion: Control flow statements manage program decisions, while loops repeat tasks efficiently in Dart.